



TradePulse Mobile App – Technical Specification Document

Version: 1.0

Date: September 2026

Platform: Flutter (iOS + Android)

Status: Draft for Development

1. Executive Summary

TradePulse is a mobile-first business management platform for informal traders, spaza shops, and township-based MSMEs in South Africa. The Flutter mobile app will serve as the primary interface for traders to record sales, manage inventory, track business performance, and generate business-readiness profiles.

Key Objectives

- Provide a **zero-friction, voice-first interface** for recording business transactions
- Enable **offline-first functionality** for areas with unreliable connectivity
- Generate **verified business records** for access to finance and supplier opportunities
- Support **multi-stakeholder access** (traders, distributors, banks, government)
- Ensure **POPIA compliance** and enterprise-grade security

Success Metrics

- 70%+ trader activation rate (3+ transactions in first 7 days)
- 90%+ transaction extraction accuracy
- <2 second app cold-start time
- 60%+ 30-day retention rate
- <1% data loss in offline scenarios

2. Target Users

Primary Users

- **Informal traders** (spaza shops, street vendors, home-based businesses)
- Age: 18–65
- Smartphone: Android (85%), iOS (15%)
- Connectivity: Intermittent 3G/4G, frequent offline periods
- Digital literacy: Low to moderate
- Languages: English, isiZulu, isiXhosa, Sesotho, Afrikaans

Secondary Users

- **Distributors:** View retailer demand, stock levels, order patterns
- **Banks/Lenders:** Access verified trading history for credit decisions
- **ESD Programmes:** Monitor supported businesses, measure impact
- **Government:** Track programme effectiveness, informal economy data

3. Functional Requirements

3.1 Core Features (MVP)

FR-001: User Registration & Authentication

- Phone number + OTP verification
- Business profile setup (shop name, location, type)
- PIN or biometric authentication (fingerprint/face ID)
- Session management with secure token storage

FR-002: Transaction Recording

- Voice note recording (up to 60 seconds)
- Text input for transaction description
- Real-time speech-to-text transcription
- AI-powered transaction extraction (product, quantity, price, type)
- Manual correction and confirmation workflow
- Transaction history with search and filter

FR-003: Inventory Management

- Product catalogue creation and management
- Automatic stock updates based on transactions
- Low-stock alerts and notifications
- Bulk product import (CSV/photo OCR – future phase)
- Product categorisation (food, beverages, household, etc.)

FR-004: Business Dashboard

- Daily/weekly/monthly sales summaries
- Revenue and profit tracking
- Best-selling products
- Stock level overview
- Transaction trends (charts and graphs)
- Export reports (PDF/CSV)

FR-005: Offline Mode

- Full offline transaction recording
- Local database sync queue
- Automatic background sync when online
- Conflict resolution for duplicate entries
- Offline dashboard access (cached data)

FR-006: Notifications & Alerts

- Low-stock push notifications
- Daily/weekly summary reminders
- Partner messages (distributor promotions, bank offers)
- App updates and maintenance notices
- Configurable notification preferences

FR-007: Readiness Passport

- Generate business profile PDF
- Trading history summary (3/6/12 months)
- Revenue trends and consistency score
- Stock turnover metrics
- Compliance indicators (optional: tax, food safety)

- Share with partners (banks, distributors, ESD)

3.2 Advanced Features (Post-MVP)

FR-008: Multi-language Support

- In-app language switching
- Voice recognition for isiZulu, isiXhosa, Sesotho, Afrikaans
- Localised product names and terminology
- Translated UI and help content

FR-009: Partner Portal Integration

- Distributor dashboard (aggregated retailer data)
- Bank/funder access (with trader consent)
- ESD programme monitoring
- Government reporting tools
- Role-based access control

FR-010: Financial Services

- Integration with payment providers (PayFast, Yoco, Ozow)
- Working capital loan applications
- Invoice generation and sharing
- Expense tracking and categorisation
- Tax estimation and reporting

FR-011: Supplier Marketplace (Future)

- Browse wholesale suppliers
- Compare prices and place orders
- Delivery tracking
- Group buying for better prices
- Supplier ratings and reviews

4. Non-Functional Requirements

4.1 Performance

Metric	Target	Measurement
Cold start time	<2 seconds	Time from app icon tap to interactive UI
Warm start time	<1 second	Time from background to foreground
Voice recording latency	<500ms	Time from button press to recording start
Transaction processing	<3 seconds	Time from submission to confirmation
Offline sync	<10 seconds	Time to sync 100 transactions on 4G
App size	<25 MB	Download size on Play Store/App Store
Memory usage	<150 MB	RAM consumption during active use

4.2 Security

Requirement	Implementation
Data encryption	AES-256 for local storage, TLS 1.3 for API
Authentication	OTP + biometric/PIN, JWT tokens with 24h expiry
Secure storage	Flutter secure storage plugin, encrypted SharedPreferences
API security	OAuth 2.0, rate limiting, input validation
POPIA compliance	Consent management, data minimisation, right to deletion
Audit logging	All transactions, logins, and data exports logged
Vulnerability scanning	Monthly dependency checks, annual penetration testing

4.3 Reliability

Metric	Target
App crash rate	<0.5% of sessions
Data loss	<0.1% of offline transactions
API uptime	99.5% (excluding scheduled maintenance)
Sync success rate	>99% of queued transactions
Error recovery	Automatic retry with exponential backoff

4.4 Scalability

Component	Initial Capacity	Scale Target
Concurrent users	1,000	50,000
Daily transactions	10,000	500,000
Database size	10 GB	500 GB

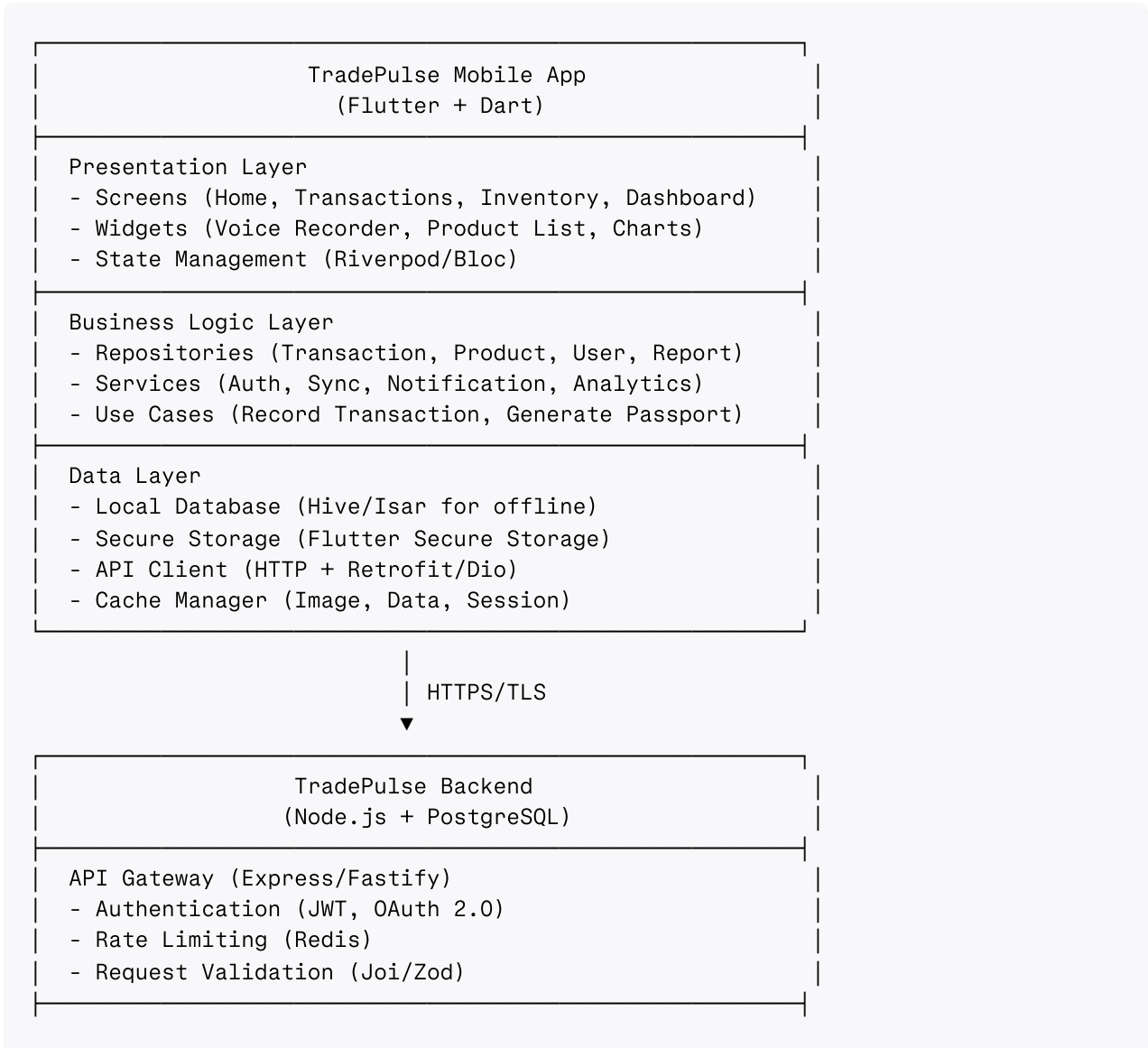
Component	Initial Capacity	Scale Target
Voice storage	100 GB	5 TB
API requests/minute	1,000	20,000

4.5 Accessibility

- WCAG 2.1 AA compliance for UI
- Screen reader support (TalkBack, VoiceOver)
- High-contrast mode for low vision
- Large text options
- Voice command support (future)

5. Technical Architecture

5.1 High-Level Architecture



```
| Core Services
| - Transaction Parser (AI + Validation)
| - Speech-to-Text (Whisper/Google)
| - Sync Engine (Queue + Conflict Resolution)
| - Notification Service (Firebase Cloud Messaging)
| - Report Generator (PDF/CSV)
```

```
| Data Storage
| - PostgreSQL (Transactions, Users, Products)
| - Redis (Cache, Queue, Sessions)
| - S3/R2 (Voice Files, Reports, Images)
| - Elasticsearch (Search, Analytics)
```

5.2 Flutter Architecture

State Management: Riverpod (recommended) or Bloc

Architecture Pattern: Clean Architecture (Presentation → Domain → Data)

Dependency Injection: GetIt or Riverpod providers

Directory Structure

```
lib/
├── main.dart
├── core/
│   ├── constants/
│   ├── errors/
│   ├── network/
│   ├── utils/
│   └── theme/
├── features/
│   ├── auth/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   ├── transactions/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   ├── inventory/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   ├── dashboard/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   └── passport/
│       ├── data/
│       ├── domain/
│       └── presentation/
├── shared/
│   ├── widgets/
│   └── models/
```

```
| └─ services/  
└─ injection_container.dart
```

5.3 Data Flow

Online Transaction Flow

```
graph TD  
    A[User records voice note] --> B[App records audio (local file)]  
    B --> C[Audio uploaded to backend (S3/R2)]  
    C --> D[Backend transcribes (Whisper API)]  
    D --> E[AI extracts transaction (GPT-4o-mini)]  
    E --> F[Validation layer checks schema + business rules]  
    F --> G[Transaction saved to PostgreSQL]  
    G --> H[Confirmation sent to app]  
    H --> I[App updates local database + UI]
```

Offline Transaction Flow

```
graph TD  
    A[User records voice note] --> B[App records audio (local file)]  
    B --> C[Transaction saved to local database (Hive/Isar)]  
    C --> D[Sync queue created with timestamp + unique ID]  
    D --> E[User continues working offline]  
    E --> F[When online: background sync service activates]  
    F --> G[Queued transactions uploaded to backend]  
    G --> H[Backend processes and confirms]  
    H --> I[Local database updated with server IDs]  
    I --> J[Sync queue cleared]
```

6. Technology Stack

6.1 Mobile App (Flutter)

Component	Technology	Version
Framework	Flutter	3.24+
Language	Dart	3.5+
State Management	Riverpod	2.4+
Local Database	Isar or Hive	Latest
Secure Storage	flutter_secure_storage	5.1+
HTTP Client	Dio or Retrofit	5.3+
Voice Recording	record + audioplayers	Latest
Push Notifications	firebase_messaging	14.6+
Biometric Auth	local_auth	2.1+
File Picker	file_picker	6.1+
Image Picker	image_picker	1.0+
PDF Generation	pdf + printing	6.0+
Charts	fl_chart or charts_flutter	Latest
Analytics	firebase_analytics	10.6+
Crash Reporting	firebase_crashlytics	3.4+
Deep Linking	app_links or uni_links	Latest

6.2 Backend Services

Component	Technology	Provider/Version
Runtime	Node.js	20 LTS
Framework	Express or Fastify	4.x / 4.x
Database	PostgreSQL	15+ (managed: Supabase/Railway)
Cache/Queue	Redis	7+ (managed: Upstash/Redis Cloud)
File Storage	AWS S3 or Cloudflare R2	Standard
AI/ML	OpenAI API (GPT-4o-mini, Whisper)	Latest
Speech-to-Text	OpenAI Whisper or Google Cloud Speech	Latest
Authentication	NextAuth.js or Auth0	Latest
Push Notifications	Firebase Cloud Messaging	Latest
Monitoring	Sentry + Datadog	Latest

Component	Technology	Provider/Version
CI/CD	GitHub Actions + Fastlane	Latest
Hosting	Render, Railway, or AWS	N/A

6.3 DevOps & Infrastructure

Component	Technology
Version Control	GitHub
CI/CD	GitHub Actions, Fastlane
App Distribution	Google Play Console, Apple App Store Connect
Beta Testing	Firebase App Distribution
Monitoring	Sentry, Datadog, Firebase Performance
Logging	Winston + Datadog Logs
Secrets Management	GitHub Secrets + AWS Secrets Manager
Database Backups	Automated daily (PostgreSQL)
Disaster Recovery	Multi-region S3 replication

7. Data Models

7.1 Core Entities

User

```
class User {
  String id;
  String phoneNumber;
  String businessName;
  String location;
  String businessType;
  DateTime createdAt;
  DateTime lastActiveAt;
  bool isVerified;
  List<String> consentFlags;
}
```

Transaction

```
class Transaction {
  String id;
  String userId;
  String type; // sale, purchase, adjustment
}
```

```

String productId;
int quantity;
double unitPrice;
double totalAmount;
DateTime timestamp;
String? voiceNoteUrl;
String? transcription;
bool isSynced;
String? serverId;
DateTime localCreatedAt;
}

```

Product

```

class Product {
    String id;
    String userId;
    String name;
    String category;
    int currentStock;
    int minStockThreshold;
    double averagePrice;
    DateTime lastUpdated;
    bool isLowStock;
}

```

BusinessReport

```

class BusinessReport {
    String id;
    String userId;
    DateTime periodStart;
    DateTime periodEnd;
    double totalRevenue;
    double totalExpenses;
    double profit;
    int totalTransactions;
    Map<String, int> topProducts;
    double consistencyScore;
    DateTime generatedAt;
    String pdfUrl;
}

```

7.2 Local Database Schema (Isar/Hive)

```

@collection
class LocalTransaction {
    Id id;
    @Index() String userId;
    String type;
    String productName;
}

```

```

    int quantity;
    double unitPrice;
    double totalAmount;
    DateTime timestamp;
    String? voiceNotePath; // Local file path
    bool isSynced;
    String? serverId;
    DateTime syncedAt;
}

@collection
class SyncQueue {
    Id id;
    String transactionId;
    String operation; // create, update, delete
    Map<String, dynamic> data;
    int retryCount;
    DateTime createdAt;
    DateTime? lastAttemptAt;
    String? errorMessage;
}

```

8. API Specifications

8.1 Authentication Endpoints

POST /api/v1/auth/register

```

Request:
{
  "phoneNumber": "+27123456789",
  "businessName": "Thandi's Spaza",
  "location": "Soweto",
  "businessType": "spaza_shop"
}

Response:
{
  "userId": "usr_123abc",
  "otpSent": true,
  "expiresIn": 300
}

```

POST /api/v1/auth/verify-otp

```

Request:
{
  "userId": "usr_123abc",
  "otp": "123456"
}

```

Response:

```
{
  "accessToken": "eyJhbGc...",
  "refreshToken": "eyJhbGc...",
  "user": { ... }
}
```

8.2 Transaction Endpoints

POST /api/v1/transactions

Request:

```
{
  "type": "sale",
  "voiceNoteUrl": "https://s3.../audio123.wav",
  "manualEntry": {
    "productName": "kota",
    "quantity": 4,
    "unitPrice": 35.00
  }
}
```

Response:

```
{
  "transactionId": "txn_456def",
  "extracted": {
    "productName": "kota",
    "quantity": 4,
    "unitPrice": 35.00,
    "totalAmount": 140.00
  },
  "requiresConfirmation": false
}
```

GET /api/v1/transactions?from=2026-09-01&to=2026-09-30

Response:

```
{
  "transactions": [ ... ],
  "totalRevenue": 15400.00,
  "totalTransactions": 127
}
```

8.3 Sync Endpoint

POST /api/v1/sync

Request:

```
{
  "transactions": [
    {
      "localId": "local_789",
      "type": "sale",
      "productName": "bread",
      "quantity": 6,
      "unitPrice": 15.00,
      "timestamp": "2026-09-16T10:30:00Z"
    }
  ],
  "lastSyncAt": "2026-09-16T08:00:00Z"
}
```

Response:

```
{
  "syncedTransactions": [
    {
      "localId": "local_789",
      "serverId": "txn_abc123",
      "status": "success"
    }
  ],
  "conflicts": [],
  "nextSyncAt": "2026-09-16T12:00:00Z"
}
```

9. Security Implementation

9.1 Authentication & Authorisation

- **OTP-based registration:** Phone number verification via SMS (Twilio/MessageBird)
- **JWT tokens:** Access token (24h expiry) + refresh token (7 days)
- **Biometric auth:** Fingerprint/face ID for app access (optional)
- **Session management:** Single active session per device, revoke on logout
- **Role-based access:** Trader, distributor, bank, admin roles

9.2 Data Protection

- **Encryption at rest:** AES-256 for local database (Isar encryption)
- **Encryption in transit:** TLS 1.3 for all API communication
- **Secure storage:** Flutter secure storage for tokens and sensitive data

- **Voice file security:** Encrypted S3 buckets, signed URLs, auto-deletion after 30 days
- **POPIA compliance:** Explicit consent for data collection, right to deletion, data portability

9.3 API Security

- **Rate limiting:** 100 requests/minute per user, 1000 requests/minute per IP
- **Input validation:** Joi/Zod schemas for all request bodies
- **SQL injection prevention:** Parameterised queries, ORM (Prisma/Sequelize)
- **CORS policy:** Whitelist app domains only
- **Audit logging:** All API calls logged with user ID, timestamp, IP

9.4 App Security

- **Certificate pinning:** Prevent MITM attacks on API calls
- **Root/jailbreak detection:** Warn or block on compromised devices
- **Code obfuscation:** ProGuard (Android), Bitcode (iOS)
- **Dependency scanning:** Monthly `flutter pub audit` + Snyk/GitHub Dependabot
- **Secure deep links:** Validate incoming URLs, prevent open redirects

10. Offline-First Strategy

10.1 Local Database

- **Technology:** Isar (recommended) or Hive for NoSQL local storage
- **Schema:** Mirror of server database with offline-specific fields
- **Encryption:** Enabled with user-specific key
- **Size limit:** 500 MB max (auto-cleanup of old data)

10.2 Sync Engine

Sync Triggers

- App comes to foreground
- Network connectivity restored
- Manual sync button pressed
- Scheduled background sync (every 4 hours)

Conflict Resolution

- **Last-write-wins:** Server timestamp takes precedence
- **Manual merge:** For critical conflicts (e.g., stock adjustments)
- **Audit trail:** All conflicts logged for review

Sync Queue

```
class SyncOperation {  
  String id;  
  String entityType; // transaction, product, user  
  String operation; // create, update, delete  
  Map<String, dynamic> payload;  
  int retryCount;  
  DateTime createdAt;  
  DateTime? lastAttemptAt;  
  String? error;  
}
```

10.3 Offline UX

- **Visual indicators:** Show "Offline" badge, sync status
- **Graceful degradation:** Disable features requiring server (e.g., passport generation)
- **Queue visibility:** Show pending sync items count
- **Error handling:** Retry failed syncs with exponential backoff
- **Data limits:** Warn user if offline database exceeds 400 MB

11. Testing Strategy

11.1 Unit Testing

- **Coverage target:** 80%+ of business logic
- **Tools:** flutter_test, mockito, bloc_test
- **Focus areas:**
 - Transaction parsing logic
 - Sync conflict resolution
 - Validation rules
 - State management

11.2 Integration Testing

- **Tools:** integration_test, patrol
- **Scenarios:**
 - Full transaction flow (record → transcribe → save → sync)
 - Offline → online transition
 - Authentication flow
 - Push notification handling

11.3 End-to-End Testing

- **Tools:** Appium, Maestro
- **Device coverage:** 10+ Android devices (low-end to high-end), 5+ iOS devices
- **Network conditions:** 3G, 4G, WiFi, offline, flaky connection
- **User scenarios:** 20+ critical user journeys

11.4 Performance Testing

- **Tools:** Flutter DevTools, Firebase Performance Monitoring
- **Metrics:**
 - App start time (cold/warm)
 - Frame rate (60 fps target)
 - Memory usage (<150 MB)
 - Battery consumption
 - Network data usage

11.5 Security Testing

- **Tools:** OWASP ZAP, Burp Suite, MobSF
- **Tests:**
 - Penetration testing (annual)
 - Dependency vulnerability scanning (monthly)
 - OWASP Mobile Top 10 compliance
 - Data leakage tests
 - Certificate pinning validation

12. Deployment & Release Management

12.1 CI/CD Pipeline

```
# GitHub Actions Workflow
name: Build & Deploy

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: subosito/flutter-action@v2
      - run: flutter pub get
      - run: flutter test --coverage
      - run: flutter analyze

  build-android:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: subosito/flutter-action@v2
      - run: flutter build apk --release
      - uses: actions/upload-artifact@v3
      with:
        name: app-release.apk
        path: build/app/outputs/flutter-apk/app-release.apk

  build-ios:
    needs: test
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v3
      - uses: subosito/flutter-action@v2
      - run: flutter build ios --release
      - uses: actions/upload-artifact@v3
      with:
        name: app-release.ipa
        path: build/ios/iphoneos/Runner.app

  deploy-beta:
    needs: [build-android, build-ios]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v3
      - uses: firebase/firebase-app-distribution-action@v1
      with:
```

```
apps: ${{ secrets.FIREBASE_APP_ID }}
token: ${{ secrets.FIREBASE_TOKEN }}
```

12.2 Release Channels

Channel	Purpose	Audience	Frequency
Dev	Daily builds	Internal team	Continuous
Alpha	Feature testing	Internal + trusted users	Weekly
Beta	Pre-release testing	100–500 pilot users	Bi-weekly
Production	Public release	All users	Monthly

12.3 App Store Compliance

Google Play Store

- Target API level 34+ (Android 14)
- Privacy policy URL required
- Content rating questionnaire
- Data safety form (data collection, sharing, security)
- APK signing with upload key

Apple App Store

- iOS 15.0+ minimum support
- App Store Connect metadata (description, screenshots, keywords)
- Privacy policy URL required
- App Review Guidelines compliance
- In-app purchase setup (if applicable)

12.4 Rollout Strategy

- **Phase 1:** 10% of users (monitor crash rate, performance)
- **Phase 2:** 50% of users (if Phase 1 stable)
- **Phase 3:** 100% rollout (after 48 hours with no critical issues)
- **Rollback plan:** Immediate revert to previous version if crash rate >1%

13. Monitoring & Analytics

13.1 Crash Reporting

- **Tool:** Firebase Crashlytics
- **Tracking:**
 - Uncaught exceptions
 - ANR (Application Not Responding)
 - Native crashes (iOS/Android)
 - Custom error logs
- **Alerts:** Slack/Email notification for critical crashes

13.2 Performance Monitoring

- **Tool:** Firebase Performance Monitoring
- **Metrics:**
 - App start time (cold/warm)
 - Network request latency
 - Screen render time
 - Custom traces (transaction recording, sync)
- **Dashboards:** Real-time performance dashboard

13.3 User Analytics

- **Tool:** Firebase Analytics + Mixpanel
- **Events:**
 - User registration
 - Transaction recorded
 - Voice note used
 - Dashboard viewed
 - Passport generated
 - Sync completed
 - Notification opened
- **Funnels:** Onboarding funnel, transaction flow, retention cohorts

13.4 Business Metrics

- **Daily Active Users (DAU)**
- **Monthly Active Users (MAU)**
- **Transactions per user per day**
- **Voice note usage rate**
- **Offline sync success rate**
- **30-day retention rate**
- **Readiness Passport generation rate**

14. Maintenance & Support

14.1 Bug Triage

Severity	Response Time	Resolution Time
Critical (app crash, data loss)	2 hours	24 hours
High (feature broken, major UX issue)	8 hours	3 days
Medium (minor bug, cosmetic issue)	24 hours	1 week
Low (enhancement request)	1 week	Next sprint

14.2 Update Cadence

- **Minor updates:** Every 2–4 weeks (bug fixes, small improvements)
- **Major updates:** Every 3–6 months (new features, UI refresh)
- **Security patches:** As needed (immediate deployment)
- **OS compatibility:** Update within 3 months of major iOS/Android release

14.3 User Support

- **In-app support:** Help center, FAQ, contact form
- **Email support:** support@tradepulse.co.za (48h response target)
- **WhatsApp support:** +27 XXX XXX XXXX (business hours)
- **Community:** Facebook/WhatsApp group for peer support
- **Feedback loop:** Monthly user surveys, feature voting

15. Risk Management

15.1 Technical Risks

Risk	Probability	Impact	Mitigation
AI extraction fails at scale	Medium	High	Human-in-the-loop validation, retry logic
Offline sync conflicts	High	Medium	Robust conflict resolution, audit trail
Voice transcription inaccurate	Medium	Medium	Manual correction workflow, multi-language models
App performance degrades	Low	Medium	Performance testing, code profiling, lazy loading
Security breach	Low	Critical	Encryption, regular audits, bug bounty program

15.2 Business Risks

Risk	Probability	Impact	Mitigation
Low trader adoption	Medium	High	Partner with distributors, incentives, training
Distributors don't pay	Medium	High	Pilot LOIs, prove ROI, flexible pricing
Competitor launches similar app	Medium	Medium	Focus on local knowledge, partnerships, data moat
Regulatory changes (POPIA)	Low	High	Legal review, compliance officer, privacy by design
Funding runs out	Medium	Critical	Milestone-based fundraising, revenue from day 1

16. Timeline & Milestones

Phase 1: MVP (Months 1–3)

- **Week 1–2:** Requirements finalisation, UI/UX design
- **Week 3–6:** Core app development (auth, transactions, inventory)
- **Week 7–8:** Backend API, database, AI integration
- **Week 9–10:** Testing (unit, integration, E2E)
- **Week 11–12:** Beta launch with 20–50 traders

Phase 2: Scale (Months 4–6)

- **Month 4:** Offline mode, sync engine, push notifications
- **Month 5:** Dashboard improvements, Readiness Passport
- **Month 6:** Partner dashboards, multi-language support
- **Milestone:** 200–500 active traders

Phase 3: Growth (Months 7–12)

- Month 7–8: Advanced analytics, financial services integration
- Month 9–10: Supplier marketplace (beta)
- Month 11–12: Regional expansion, iOS optimisation
- Milestone: 1,000+ active traders, 3+ distributor contracts

17. Budget Estimate

Development Costs

Component	Cost (ZAR)
Flutter app development (6 months)	R900,000 – R1,500,000
Backend development (6 months)	R600,000 – R1,000,000
UI/UX design	R150,000 – R300,000
Project management	R200,000 – R400,000
Testing & QA	R150,000 – R300,000
Total Development	R2,000,000 – R3,500,000

Infrastructure Costs (Monthly)

Component	Cost (ZAR)
Cloud hosting (backend)	R5,000 – R20,000
Database (PostgreSQL)	R3,000 – R10,000
File storage (S3/R2)	R2,000 – R8,000
AI/ML (OpenAI, Whisper)	R10,000 – R50,000
Push notifications (Firebase)	R0 – R2,000
Monitoring (Sentry, Datadog)	R3,000 – R10,000
Total Monthly	R23,000 – R100,000

Operational Costs (Monthly)

Component	Cost (ZAR)
Customer support (2 agents)	R30,000 – R60,000
Marketing & user acquisition	R20,000 – R100,000
Legal & compliance	R10,000 – R30,000
Contingency (15%)	R9,000 – R28,500

Component	Cost (ZAR)
Total Monthly	R69,000 – R218,500

18. Success Criteria

Technical Success

- ✔ App launches in <2 seconds
- ✔ 99.5% API uptime
- ✔ <0.5% crash rate
- ✔ 99% sync success rate
- ✔ 80%+ test coverage

Business Success

- ✔ 500+ active traders in 6 months
- ✔ 70%+ activation rate
- ✔ 60%+ 30-day retention
- ✔ 3+ paying distributor contracts
- ✔ R500,000+ monthly recurring revenue in 12 months

19. Approval & Sign-off

Role	Name	Signature	Date
Product Owner			
Technical Lead			
Project Manager			
Security Officer			
Stakeholder			

Document Control:

- **Version:** 1.0
- **Status:** Draft
- **Next Review:** October 2026
- **Distribution:** Development Team, Stakeholders, Investors

This technical specification provides a comprehensive blueprint for building the TradePulse Flutter mobile app. It balances technical rigor with practical implementation details, ensuring

the development team has clear guidance while allowing flexibility for iteration and improvement.